

2026년도 전국기능경기대회 채점기준

1. 채점상의 유의사항	직 종 명	클라우드 컴퓨팅
※ 다음 사항을 유의하여 채점하시오. 1) AWS의 리전은 ap-northeast-2(서울)을 사용합니다. 2) 모든 채점은 CloudShell에서 진행하며, Bastion EC2는 사용하지 않습니다. 3) 채점 명령어 실행은 CloudShell의 IAM User(Role) 권한으로 실행합니다. IAM User의 AccessKey로 실행하지 않으며, 구성된 경우 삭제 후 진행합니다. 4) 채점 스크립트(1과제_채점스크립트_ver1_0_3.sh)를 활용하며, 선수가 이의를 제기하는 경우 해당 명령어를 직접 실행하여 확인합니다. 5) 문제지와 채점지에 있는 <> 는 변수입니다. 해당 부분을 적절한 값으로 변경하여 입력합니다. 6) 채점은 문항 순서대로 진행합니다. 7) 부분 점수가 있는 문항은 채점 항목에 배점이 명시되어 있습니다. 8) 부분 점수가 없는 문항은 조건을 모두 만족하여야 점수로 인정됩니다. 9) 채점 명령어는 여러 번 실행하여도 동일한 결과를 보장합니다. 단, 6-2 항목은 타임스탬프 기반 고유 client_id를 사용하므로 반복 실행 시 PASS/FAIL 판정은 동일하게 유지됩니다. 10) 문제 요구사항 외 불필요한 리소스가 확인될 경우 8-2 항목은 0점 처리합니다. 11) 채점 후 이의신청이 완료되면 선수가 생성한 클라우드 리소스를 삭제합니다. 12) [] 기호는 채점에 영향을 주지 않습니다. 13) 쉘 명령어의 \$ 기호는 쉘 프롬프트를 의미하며 명령어에 포함되지 않습니다. 14) <선수ID>는 선수의 비번호를 의미합니다. AWS Account ID가 아니며, 모든 리소스 이름 및 Name Tag에서 <선수ID>로 표시된 부분은 본인의 비번호로 치환하여 사용합니다.		

2. 채점기준표

1) 주요항목별 배점			직 종 명		클라우드 컴퓨팅			
과제 번호	일련 번호	주요항목	배점	채점방법		채점시기		비고
				독립	합의	경기 진행중	경기 종료후	
1과제	1	네트워크 (VPC/Subnet/IGW)	4	○			○	
	2	정적 웹 호스팅 (S3/CloudFront)	4	○			○	
	3	ECR	2.5	○			○	
	4	ECS/Fargate	5.5	○			○	
	5	ALB	3	○			○	
	6	DynamoDB	4	○			○	
	7	CloudWatch Logs	4	○			○	
	8	종합 동작 확인	3	○			○	
합 계			30					

2) 채점방법 및 기준

(경기종료 후 채점)

과제 번호	일련 번호	주요항목	일련 번호	세부항목(채점방법)	배점
1과제	1	네트워크 (VPC/Subnet/IGW)	1	VPC 및 Public Subnet 확인	1.5
			2	Internet Gateway 및 라우팅 확인	1.5
			3	리소스 명명 규칙 확인	1
	2	정적 웹 호스팅	1	S3 파일 업로드 확인	1.5
			2	S3 퍼블릭 접근 차단 및 OAC/OAI 구성 확인	1
			3	CloudFront URL 접근 및 Distribution 상태 확인	1.5
1과제	3	ECR (Elastic Container Registry)	1	ECR Repository 생성 확인	1
			2	ECR 이미지 업로드 및 Linux/AMD64 확인	1.5
1과제	4	ECS/Fargate	1	ECS Cluster 및 Task Running 확인	1
			2	Task Definition 설정 확인	1.5
			3	환경 변수 설정 확인	1
			4	IAM Role 및 CPU/MEM 설정 확인	1
			5	헬스 체크 확인	1
1과제	5	ALB (Application Load Balancer)	1	ALB 생성 및 ECS 연결 확인	1
			2	Listener, Target Group 및 Target Health 확인	1.5
			3	보안그룹 규칙 확인	0.5
1과제	6	DynamoDB	1	DynamoDB 테이블 생성 확인	1
			2	예약 API 호출 및 DynamoDB 저장 확인	1.5
			3	저장 데이터 스키마 정합성 확인	1.5
1과제	7	CloudWatch Logs	1	로그 그룹 생성 확인	1
			2	로그 스트림 생성 및 로그 수집 확인	1.5
			3	awslogs 드라이버 설정 확인	1.5
1과제	8	종합 동작 확인	1	전체 연계 동작 최종 확인	1.5
			2	불필요 리소스 미존재 확인	1.5

3) 채점내용

순번	사전 준비
0	1) 채점 전 IAM User의 AccessKey가 구성된 경우 삭제 후 진행합니다. 2) CloudShell을 실행하고 리전이 ap-northeast-2(서울)인지 확인합니다. 3) 채점 스크립트(grade_task1.sh)를 CloudShell에 업로드한 후 실행합니다. \$ bash grade_task1.sh <선수ID> 4) ECS Task가 Running 상태가 될 때까지 최대 3분 대기합니다. 5) 지급 파일(index.html, main.jpeg, book) 사용 여부를 확인합니다.
1-1	VPC(CIDR 10.0.0.0/16) 및 Public Subnet 2개(서로 다른 AZ) 확인 \$ aws ec2 describe-vpcs --filters "Name=tag:Name,Values=<선수ID>-vpc" "Name=cidr-block,Values=10.0.0.0/16" --query "Vpcs[*].{ID:VpcId,CIDR:CidrBlock}" --output table \$ aws ec2 describe-subnets --filters "Name=tag:Name,Values=<선수ID>-public-subnet-*" --query "Subnets[*].{ID:SubnetId,AZ:AvailabilityZone}" --output table → VPC 1개(10.0.0.0/16), Subnet 2개 이상(서로 다른AZ) 확인 시 PASS
1-2	Internet Gateway VPC 연결 및 0.0.0.0/0 → IGW 라우팅 확인 \$ aws ec2 describe-internet-gateways --filters "Name=attachment.vpc-id,Values=<VPC-ID>" --query "InternetGateways[*].{IGW:InternetGatewayId,State:Attachments[0].State}" --output table \$ aws ec2 describe-route-tables --filters "Name=vpc-id,Values=<VPC-ID>" --query "RouteTables[*].Routes[?DestinationCidrBlock=='0.0.0.0/0']" --output table → IGW Attached, 0.0.0.0/0 → IGW 경로 존재 시 PASS
1-3	VPC, Subnet, IGW, Route Table 이름에 선수ID 접두어 포함 확인 \$ aws ec2 describe-vpcs --filters "Name=tag:Name,Values=<선수ID>-*" --query "Vpcs[*].Tags[?Key=='Name'].Value" --output table → <선수ID>-vpc, <선수ID>-public-subnet-1/2 등 확인 시 PASS
2-1	index.html, main.jpeg를<선수ID>-static-site 버킷에 업로드 확인 \$ aws s3 ls s3://<선수ID>-static-site/ --region ap-northeast-2 → index.html, main.jpeg 모두 존재 시 PASS
2-2	S3 퍼블릭 접근 차단(Block Public Access) 및 CloudFront OAC/OAI 구성 확인 \$ aws s3api get-public-access-block --bucket <선수ID>-static-site --query "PublicAccessBlockConfiguration.BlockPublicAcls" --output text → True 반환 시 PASS

순번	사전 준비
2-3	1) CloudFront URL로 index.html HTTP 200 접근 확인(1점) <pre>\$ CF=\$(aws cloudfront list-distributions \ --query "DistributionList.Items[?Origins.Items[?contains(DomainName,'<선수ID>-static-site')]].DomainName [0]" \ --output text) \$ curl -s -o /dev/null -w "%{http_code}" "https://\$CF/"</pre> → 200 반환 시 PASS <pre>\$ CF_ID=\$(aws cloudfront list-distributions \ --query "DistributionList.Items[?Origins.Items[?contains(DomainName,'<선수ID>-static-site')]].Id [0]" \ --output text) \$ aws cloudfront get-distribution --id \$CF_ID \ --query "Distribution.DistributionConfig.DefaultRootObject" \ --output text</pre> → index.html 반환 시 PASS 2) Distribution 상태 확인(0.5점) <pre>\$ aws cloudfront get-distribution --id <Distribution-ID> \ --query "Distribution.Status" --output text</pre> → Deployed 반환 시 PASS
3-1	<선수ID>-book-ecr ECR 프라이빗 Repository 생성 확인 <pre>\$ aws ecr describe-repositories --repository-names <선수ID>-book-ecr --query "repositories[0].repositoryUri" --output text</pre> → URI 반환 시 PASS
3-2	1) ECR에 latest 태그 이미지 존재 확인(1점) <pre>\$ aws ecr describe-images --repository-name <선수ID>-book-ecr --query "imageDetails[?imageTags[?@=='latest']].{Tags:imageTags,Pushed:imagePushedAt}" --output table</pre> → latest 태그 이미지 존재 시 PASS 2) Linux/AMD64 이미지 확인 (0.5점) <pre>\$ aws ecs describe-task-definition \ --task-definition \$TD \ --query "taskDefinition.runtimePlatform.cpuArchitecture"</pre> → X86_64 반환 시 PASS
4-1	ECS Cluster(<선수ID>-book-cluster) ACTIVE 및 Task Running 확인 ※ 대기: 최대3분 1) CloudShell에서 아래 명령어를 실행합니다. <pre>\$ aws ecs describe-clusters --clusters <선수ID>-book-cluster --query "clusters[0].status" --output text \$ aws ecs list-tasks --cluster <선수ID>-book-cluster --desired-status RUNNING --query "length(taskArns)" --output text</pre> → ACTIVE, 1 이상 반환 시 PASS

순번	사전 준비
4-2	Task Definition 설정 확인 (1.5점) 1) CloudShell에서 아래 명령어를 실행합니다. <pre>\$ aws ecs describe-task-definition \ --task-definition \$TD \ --query "taskDefinition.{CPU:cpu,MEM:memory,ARCH:runtimePlatform.cpuArchitecture,PORT:containerDefinitions[0].portMappings[0].containerPort}" \ --output table</pre> → PORT=8080 → CPU=256 → MEM=512 → ARCH=X86_64 모두 확인 시 PASS
4-3	AWS_REGION, TABLE_NAME 환경변수 설정 확인 <pre>\$ aws ecs describe-task-definition --task-definition \$TD --query "taskDefinition.containerDefinitions[*].environment" --output table</pre> → AWS_REGION=ap-northeast-2, TABLE_NAME=<선수ID>-booking-table 확인 시 PASS
4-4	Task Execution Role(ECR pull + CW Logs) 및 Task Role(DynamoDB PutItem 이상) 설정 확인 <pre>\$ aws ecs describe-task-definition --task-definition \$TD --query "taskDefinition.{Exec:executionRoleArn,Task:taskRoleArn}" --output table</pre> → 두 Role 모두 값 존재 시 PASS
4-5	Health Check 확인 (1점) <pre>\$ CF=\$(aws cloudfront list-distributions \ --query "DistributionList.Items[?Origins.Items[?contains(DomainName,'<선수ID>-static-site')]].DomainName [0]" \ --output text) \$ curl -s -o /dev/null -w "%{http_code}" "https://\$CF/health"</pre> → 200 반환 시 PASS
5-1	ALB(<선수ID>-book-alb) Internet-facing 생성 및 ECS Service 연결 확인 <pre>\$ aws elbv2 describe-load-balancers --names <선수ID>-book-alb --query "LoadBalancers[0].{Scheme:Scheme,State:State.Code}" --output table</pre> → Scheme: internet-facing, State: active 확인 시 PASS

순번	사전 준비
5-2	Listener, Target Group 및 Target Health 확인 <pre> \$ ALB_ARN=\$(aws elbv2 describe-load-balancers \ --names <선수ID>-book-alb \ --query "LoadBalancers[0].LoadBalancerArn" \ --output text) \$ aws elbv2 describe-listeners \ --load-balancer-arn \$ALB_ARN \ --query "Listeners[*].[Port,Protocol]" \ --output table \$ TG_ARN=\$(aws elbv2 describe-target-groups \ --load-balancer-arn \$ALB_ARN \ --query "TargetGroups[0].TargetGroupArn" \ --output text) \$ aws elbv2 describe-target-groups \ --target-group-arns \$TG_ARN \ --query "TargetGroups[*].[Port,Protocol,TargetType]" \ --output table \$ aws elbv2 describe-target-health \ --target-group-arn \$TG_ARN </pre> → Listener : HTTP/80 → Target Group : HTTP/8080, IP → Healthy 대상 1개 이상 모두 확인 시 PASS
5-3	ALB SG 인바운드 HTTP:80 허용 및 ECS SG 인바운드 TCP:8080(소스: ALB SG ID) 확인 <pre> \$ aws ec2 describe-security-groups --filters \ "Name=group-name,Values=<선수ID>-alb-sg" \ --query "SecurityGroups[0].IpPermissions[?FromPort==\`80\`].IpRanges[0].CidrIp" \ --output text tr '\t' '\n' grep '0.0.0.0/0' \$ aws ec2 describe-security-groups --filters \ "Name=group-name,Values=<선수ID>-ecs-sg" \ --query "SecurityGroups[0].IpPermissions[?FromPort==\`8080\`].UserIdGroupPairs[0].GroupId" \ --output text </pre> → ALB SG ID가 ECS SG 인바운드 소스로 설정되어 있으면 PASS
6-1	테이블(<선수ID>-booking-table) ACTIVE, PK: client_id(String), Billing Mode 확인 <pre> \$ aws dynamodb describe-table --table-name <선수ID>-booking-table \ --query "Table.{Status:TableStatus,Key:KeySchema[0],Billing:BillingModeSummary.BillingMode}" \ --output table </pre> → Status:ACTIVE, AttributeName:client_id, Billing:PAY_PER_REQUEST 또는 PROVISIONED 확인 시 PASS

순번	사전 준비
6-2	POST /v1/book 호출 시 booking_id 응답 및 DynamoDB 저장 확인 ※ 멍등성: 채점 시마다 타임스탬프 기반 고유 client_id 사용 <pre> \$ TS=\$(date +%s) \$ CF=\$(aws cloudfront list-distributions \ --query "DistributionList.Items[?Origins.Items[?contains(DomainName,'<선수ID>-static-site')]].DomainName [0]" \ --output text) \$ curl -s -X POST "https://\$CF/v1/book" \ -H "Content-Type: application/json" \ -d "{\"client_id\":\"chk-<선수ID>-\$TS\", \"username\":\"채점자\", \"email\":\"chk@t est.com\", \"concert_name\":\"Seoul2026\"}" \$ aws dynamodb get-item --table-name <선수ID>-booking-table \ --key "{\"client_id\":{\"S\":\"chk-<선수ID>-\$TS\"}}" --output json → {"booking_id": "..."} 반환 및 DynamoDB Item 존재 시 PASS </pre>
6-3	DynamoDB 저장 데이터 6개 속성 정합성 확인 (client_id, booking_id, username, email, concert_name, created_at) <pre> \$ aws dynamodb get-item --table-name <선수ID>-booking-table \ --key "{\"client_id\":{\"S\":\"chk-<선수ID>-\$TS\"}}" --query "Item" --output json → 6개 속성 모두 존재 시 PASS </pre>
7-1	로그 그룹/skillskorea/ecs/app 생성 확인 1) CloudShell에서 아래 명령어를 실행합니다. <pre> \$ aws logs describe-log-groups \ --log-group-name-prefix "/skillskorea/ecs/app" \ --query "logGroups[*].logGroupName" --output text → /skillskorea/ecs/app 반환 시 PASS </pre>
7-2	7-2 로그 스트림 생성 및 로그 수집 확인 <pre> \$ aws logs describe-log-streams \ --log-group-name "/skillskorea/ecs/app" \ --query "logStreams[?starts_with(logStreamName,'ecs/')].logStreamName" \$ aws logs filter-log-events \ --log-group-name "/skillskorea/ecs/app" \ --filter-pattern "" \ --start-time \$((\$(date +%s)-3600)*1000) \ --no-paginate \ --query "length(events)" --output text head -1 → ecs/ 접두어 로그 스트림 1개 이상 존재 → 로그 이벤트 1개 이상 존재 모두 확인 시 PASS </pre>
7-3	awslogs 드라이버 설정 확인 <pre> \$ aws ecs describe-task-definition \ --task-definition \$TD \ --query "taskDefinition.containerDefinitions[0].logConfiguration" \ --output json → LogDriver=awslogs → awslogs-group=/skillskorea/ecs/app 모두 확인 시 PASS </pre>

순번	사전 준비
8-1	CloudFront→index.html(200), CloudFront→/health(200), POST /v1/book→DynamoDB 연계 최종 확인 <pre>curl -s -o /dev/null -w "CF_ROOT:%{http_code}" "https://\$CF/" curl -s -o /dev/null -w " CF_HEALTH:%{http_code}" "https://\$CF/health" curl -s -X POST "https://\$CF/v1/book" \ -H "Content-Type: application/json" \ -d "{\"client_id\": \"chk-<선수ID>-<TS>\", \"username\": \"채점자\", \"email\": \"chk@t est.com\", \"concert_name\": \"Seoul2026\"}"</pre> → CloudFront / : 200 → CloudFront /health : 200 → POST /v1/book 정상 응답 확인 시PASS
8-2	불필요 리소스 미존재 확인 <pre>\$ aws elbv2 describe-load-balancers \ --query "LoadBalancers[?LoadBalancerName!=<선수ID>-book-alb'].LoadBalancerName" \ --output text \$ aws ec2 describe-vpcs \ --filters "Name=isDefault,Values=false" \ --query "Vpcs[?CidrBlock!='10.0.0.0/16'].VpcId" \ --output text \$ aws ec2 describe-instances \ --filters "Name=instance-state-name,Values=running,stopped" \ --query "Reservations[*].Instances[*].InstanceId" \ --output text</pre> → 문제 요구사항 외 불필요한 리소스가 존재하지 않으면PASS → 미사용Load Balancer, EC2, VPC 등이 존재하면FAIL